Improved Garbled RAM via Garbled Merge *

Can Liu[†‡]
Tsinghua University
liucan22@mails.tsinghua.edu.cn

Lenny Liu[†]
UIUC
hengyu2@illinois.edu

Ning Luo
UIUC
nl27@illinois.edu

David Heath
UIUC
daheath@illinois.edu

Abstract

Consider the problem of merging inside a garbled circuit (GC) two arrays of w-bit elements, yielding a single length-n array. This garbled merge problem is core to garbled random access memory (GRAM), a technique that enables efficient garbling of general-purpose programs. We present a novel symmetric-key-based garbled merge that achieves a garbling size of $(w+1)\cdot n\cdot\lambda$ bits, providing both asymptotic and concrete improvements over the state of the art. By applying our garbled merge, we obtain a symmetric-key GRAM of size $O(n\lg^3 n\cdot\lambda)\cdot\omega(1)$ for a word RAM program that manipulates words of size $\Theta(\lg n)$ bits and halts within n steps, improving over the previous best result (Heath et al., CRYPTO'23) by an $O(\lg\lg n)$ factor. This communication cost was previously only achieved under the public-key-style DDH assumption (Gu et al., CRYPTO'25). We implement our construction, and our evaluation shows that our garbled merge reduces the communication cost over the DDH-based merge by about 3×.

Keywords: Garbled Circuits, Garbled RAM, Secure Multiparty Computation

[†]Authors contributed equally to this work.
[‡]This work was done while this author was at UIUC.
*An abridged version appears in the proceedings of ACM CCS 2026.

Contents

1 Introduction
1.1 Contribution . . .

2 Technical Overview
2.1 Switch System Review . . .
2.2 Garbled Merge as a Finite State Automaton . . .

3 Preliminaries
3.1 Basic Notation . . .
3.2 Garbling Schemes . . .
3.3 Cryptographic Assumption . . .
3.4 Tri-State GRAM; How Our Merge Plugs In . . .

4 Garbled Switch System Review

5 Garbled Merge
5.1 Merge as a Switch System . . .
5.2 Randomized Merge . . .
5.3 Improved Garbled RAM . . .

6 Evaluation
6.1 Garbled Merging Evaluation . . .
6.2 End-to-end Evaluation. . . .

A Garbling Scheme Definition

B Circular Correlation Robustness

C Improving Computation Cost for Randomized Merge

D Garbled Merge Sort

E A Switch System Garbling Scheme

1 Introduction

Secure two-party and multi-party computation (2PC and MPC) allow mutually untrusting parties to jointly compute a function over their private inputs, while revealing nothing beyond the function's output. Yao's Garbled Circuit (GC) is a fundamental primitive in cryptography, central to 2PC/MPC [Yao86]. In GC, one party, called the *garbler* (G), constructs a *garbling* of a program P, denoted $\tilde{P}$. Traditionally, P is a Boolean circuit. The garbler G conveys to the evaluator E the garbled program $\tilde{P}$, along with encryptions of the parties' inputs. E then executes the garbled program $\tilde{P}$ gate-by-gate: at each gate, E uses encodings of wire values to decrypt subsequent encodings of wire values, ultimately leading to E holding an encoding of program output. G and E then jointly decrypt that output and recover $P(x)$. GC enables 2PC/MPC protocols that use only a *constant* number of communication rounds, making the technique a good choice when network latency is high.

Improving the asymptotic and concrete cost of GC is critical for making constant-round 2PC/MPC more practical. The primary goal is reducing *communication complexity*, dictated by the size of the garbled program $\tilde{P}$. This is because communication is often the bottleneck in secure computation, particularly in bandwidth-constrained settings. Trading increased local computation for reduced communication is generally favorable, as modern processors can perform millions of cryptographic operations per second. *Space complexity*, though often overlooked, is equally important: parties may face prohibitive memory overhead from large garbled circuits and wire encodings. Notably, reducing the garbled program size does not necessarily reduce overall space, as advanced GC techniques (including ours) can produce wire encodings that asymptotically exceed the garbled program itself.

Garbled RAM While much of the GC literature focuses on secure evaluation of Boolean circuits, it is also possible to garble other computational primitives. One of the most important special-case garbling problems is that of garbled *random access memory* (GRAM) [LO13]. GRAM is essential for garbling *general-purpose* programs. Without efficient RAM access, off-the-shelf programs would incur prohibitive overhead when storing or manipulating large amounts of data in non-trivial data structures. Because of its importance, GRAM has been studied extensively, see e.g. [LO13, GHL$^+$14, GLOS15, GLO15, HKO22, PLS23, HKO23, GTRS25]. Roughly speaking, GRAM is achieved by implementing Oblivious RAM inside the garbled program [GO96]. Its efficiency is typically measured by the size of the garbling of an n-step RAM program manipulating w-bit words.

Construction	Assumption	Communication
TSC [HKO23]	OWF	$O(n(w+\lg^2 n)\lg n \cdot \lg\lg n \cdot \lambda)$
PicoGRAM [GTRS25]	DDH+RO	$O(n(w+\lg^2 n)\lg n \cdot \lambda)\cdot\omega(1)$
Ours	CCRH	$O(n(w+\lg^2 n)\lg n \cdot \lambda)\cdot\omega(1)$

Table 1: Comparing our GRAM with existing constructions.

Prior to this work, there were two incomparable state-of-the-art GRAM schemes: The GRAM of [HKO23] achieves size $O(n\cdot(w+\lg^2 n)\cdot\lg n\cdot\lg\lg n\cdot\lambda)$ bits [HKO23], which simplifies to $O(n\cdot\lg^3 n\cdot\lg\lg n\cdot\lambda)$ when $w = \Theta(\lg n)$. The [HKO23] scheme is secure assuming only one-way functions (OWFs). More recently, [GTRS25] used the DDH assumption to replace particular gates internal to the [HKO23] scheme, shaving a $\lg\lg n$ factor and achieving a GRAM of size $O(n\cdot\lg^3 n\cdot\lambda)\cdot\omega(1)$ bits. However, this comes at the cost of taking a public-key-style assumption, which can degrade runtime performance.

Thus, there was a trade-off in the literature: either use symmetric-key-based GC and accept a larger garbled program, or pay the cost of public-key-based GC (and its corresponding computation costs) in exchange for a smaller garbled program. Our work resolves this trade-off by achieving a symmetric-key GRAM with asymptotic size matching that of [GTRS25]. Our scheme is secure assuming a circular-correlation robust hash function (CCRH, see Definition 6), an assumption almost ubiquitous in the GC literature due to its importance to the celebrated Free XOR optimization [KS08].

The challenge of GRAM The crucial technical challenge[1] of GRAM is one of *routing*. To achieve random access memory, array elements must be routed from one part of the garbled program (where the memory element is stored) to another (where the element is used). Routing in GC is difficult *even when the evaluator E is told in cleartext how all data should move.* The source of this difficulty is that the garbler G *cannot* learn these routing details without breaking GC's central constant-round property. Indeed, routing decisions might not be computable until *runtime*, when the garbler G is not available.

In standard GC techniques (including ours), each wire value is encoded by a particular ciphertext. Therefore, if we consider an array x, we can encode each of its bits x_i by a distinct ciphertext, which we denote by $[x_i]$. Now, suppose G would like to prepare a garbled gate that takes as input some particular wire x_i where the array index i is decided at runtime.

To correctly garble a particular gate, G must know the relevant ciphertexts in advance. Since array index i is revealed only at runtime, G cannot anticipate which ciphertext $[x_i]$ will be needed. As a result, G cannot directly garble the gate ahead of time, despite the fact that E can easily access the correct ciphertext $[x_i]$ once i is revealed. Indeed, known techniques for GRAM include additional structure that *routes* memory elements to their eventual destinations. Such routing structures can be achieved by simple Boolean gates. However, this incurs additional garbled gates, and hence additional cost. Known Boolean-circuit-based sorting networks incur polylogarithmic blow-up in total cost that we would like to avoid. Our main contribution is a routing structure that requires asymptotically less communication.

In the GRAM literature, the routing challenge is called the *language translation problem*. Each wire has a specific *language* – the possible ciphertexts that can appear on that wire at runtime. To place a garbled value onto a particular wire, E must *translate* her encoding to the language of that wire.

Garbled Merge We study *garbled merge* as a canonical and central instance of the language problem. Garbled merge enables a *partially-private online stable merge*, in which two garbled arrays are merged into a single array according to a sequence of control bits. As an example of garbled merge, suppose x^0 is a length-2 array, x^1 is a length-3 array, and consider a string of *control bits* $s = 10110$. Then the output array y should be $x_0^1 x_0^0 x_1^1 x_2^1 x_1^0$. Notice that the control bits appear as superscripts, and elements of each respective array appear in their original relative order (the merge is *stable*). The merge is *partially-private* because we require that the control bits s be revealed to the GC evaluator E. The array elements x^0, x^1 are *not* revealed. In GRAM, the revealed control bits can be simulated, due to the GRAM's use of an underlying oblivious RAM.

Garbled merge is crucial to state-of-the-art garbled RAM. Indeed, the problem is implicitly solved in [HKO22, HKO23]. Specifically, they use two Boolean-circuit-based stacks, as formalized by [ZE13], to achieve a merge. The recent PicoGRAM construction [GTRS25] improved over [HKO23]'s GRAM by replacing its handling of garbled merge with a DDH-based solution. Similarly, we obtain improved GRAM by replacing [HKO23]'s handling of garbled merge with our own.

A baseline Boolean-circuit-based garbled merge uses $O(w \cdot n \lg n)$ gates, where w is the bit-length of array elements, leading to a garbling of size $O(w \cdot n \lg n \cdot \lambda)$ bits. The recent PicoGRAM construction [GTRS25] improves this to $O((w + \lg n) \cdot n \cdot \lambda_{DDH})$ bits, but theirs relies on a public-key assumption (DDH). Here λ_{DDH} is the size of a DDH group element.

1.1 Contribution

- **State-of-the-art garbled merge.** We present a novel online algorithm (i.e., a data structure) for garbled merge. Our construction remains in Minicrypt and achieves a garbling size of $(w + 1) \cdot n \cdot \lambda$ bits, an asymptotic and concrete improvement over the state of the art. We also show the applications of garbled merge beyond GRAM in Appendix D.

- **State-of-the-art GRAM under weak assumption.** By integrating our technique into existing GRAM [HKO23], we achieve a symmetric-key GRAM with communication cost matching that of [GTRS25]. Our scheme is secure under the assumption of a circular-correlation-robust hash function (CCRH), a common and widely adopted assumption in the GC literature [KS08].

[1] Another challenge in GRAM is obfuscating the memory's logical access pattern. GRAMs tend to leak memory access patterns to E, yet they must achieve privacy. This challenge is typically addressed by carefully integrating ORAM.

- **Space improvement.** Our G (resp. E) can garble (resp. evaluate) a merge within asymptotically less *space* ($O(w \cdot n \cdot \lambda)$ bits) than the Boolean-circuit-based baseline (requiring $\Omega(w \cdot n \lg n \cdot \lambda)$ bits of space). Low space usage is not trivially implied by the fact that the resulting garbled program is smaller. Indeed, in our merge, E will compute encodings of wire values whose joint length exceeds size of the garbled program, but E can nevertheless run in low space.
- **Modest computational cost.** The computational cost incurred in exchange for reduced communication remains modest. We show that garbled merge requires only $O(w \cdot n \cdot \sqrt{n \cdot \lambda})$ calls to a cryptographic hash function when the control string s is random. When applied to GRAM, the instantiated merge outputs a relatively small array of size $O(\lambda)$, resulting in a small additional computation cost while still achieving an asymptotic improvement in communication over the Boolean circuit baseline.
- **Implementation and evaluation.** We implement and evaluate our protocols.[2] Our evaluation shows that garbled merge outperforms PicoGRAM's compaction-network-based solution by approximately $3\times$ in communication.

2 Technical Overview

Our technical observations are two-fold:

- The stable merge of two arrays can be encoded in the execution of a particular *finite state automaton* (FSA).
- The transition behavior of our FSA can be encoded as a particularly garbling-friendly *system of constraints*.

In particular, prior work [Hea24] showed that certain systems of constraints—namely, those formalized in a model of computation called the *switch system* model—can be efficiently garbled. We encode our FSA as a garbled switch system. This switch system incurs particularly low communication cost, leading to our main result.

This section describes our techniques at a high level. We start by reviewing the switch system model and highlighting its advantages, in particular its ability to efficiently compute on *one-hot vectors*. Then, we describe our merge FSA and explain how we encode its behavior as a switch system.

2.1 Switch System Review

Since our approach is based on switch systems [Hea24], we begin by reviewing the computational model to establish common background knowledge. We focus on the model's main two gates: switches and joins. Section 4 provides further details on the switch system model. We note that the switch system model was proposed in the context of garbled arithmetic operations; our work shows it is also highly relevant to non-trivial problems in garbled data structures.

In general, any computation can be understood as solving a system of constraints (the program) over program variables, including inputs, outputs, and intermediate variables. From this perspective, Boolean circuits can be understood as a method for writing restricted forms of constraint systems. The switch system model offers a more flexible collection of primitives for expressing constraints. While Boolean circuits express constraints that must be solved from left to right (i.e., from inputs to outputs), switch systems allow greater flexibility in that their gates can express constraints in the true sense, where an evaluator may solve for variables in whichever order is convenient to the current execution. We leverage this additional expressivity in our handling of garbled merge.

More precisely, a switch system is defined over a set of variables or wires; its gates specify logical constraints on those variables. The central gate in the switch system model is called a *switch*: A switch ($_\!\!\diagup\!\!_$) takes as input a *data wire* $x \in \{0,1\}$ and a control bit $ctrl \in \{0,1\}$. It outputs a second data wire y. When $ctrl = 0$, the switch "closes" and imposes the constraint that values on the two data wires must be equal, i.e., $x := y \Leftarrow ctrl \stackrel{?}{=} 0$. From an execution perspective, if only one data wire carries a defined value while

[2] Our repository is available at: `https://zenodo.org/records/20751401`

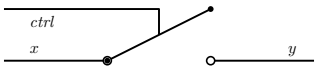


Figure 1: A switch gate.

the other is undefined, a switch with $ctrl = 0$ propagates the defined value to the undefined wire. From the broader viewpoint of the constraint system, this enforces that the value of one wire can be inferred from the other whenever the control bit is 0.

Looking ahead, garbled evaluation of a switch requires the evaluator E to know the control bit $ctrl$ in cleartext. This is because E will act conditionally depending on $ctrl$. If $ctrl$ holds zero, then E uses a hash function to "translate" between the two data wires; else, E does nothing. This introduces two points: (1) all cleartext switch controls must be computable by E, and (2) the cleartext controls must reveal no useful information about private inputs. Our garbled merge incorporates many switches, but all control values can be inferred from the merge's input control string s, and s can be simulated due to the garbled RAM's use of ORAM techniques.

A *join gate* $(x \bowtie y)$ unconditionally enforces the equivalence of its two inputs x and y. From an execution perspective, if one wire is defined while the other is undefined, the join gate sets the undefined wire to the defined value. From the perspective of the constraint system, this ensures that the value of either wire can always be inferred from the other as soon as one becomes defined.

Crucially, join gates are the only gates that incur significant communication when garbled. Switches, for instance, require no communication. For the unfamiliar reader, it may be unclear why joins are needed, since they simply express an equality constraint. Join gates are necessary due to the syntactic rules enforced by the switch system model. Suppose, for instance, we would like to instantiate two switches whose output data wires are, in fact, the same wire. Such an arrangement could be used to implement the following conditional assignment:

$$\text{if } ctrl = 0 \text{ then } z := x \text{ else } z := y$$

The switch system model explicitly disallows expressing this conditional as two lone switches. Indeed, each switch outputs a distinct, fresh wire. That is, there is a difference in how wires are treated when the switch system is *specified* (wires are created by specifying gates from left to right) from how they are *evaluated* (gates specify multidirectional constraints). Looking ahead, this duality reflects the fact that the garbler handles switch systems in one way (he garbles, left to right), and the evaluator handles them in another (she evaluates, in whichever order computes the output).

If we wish to state that two wires are "the same", we can do so by means of a join. The insistence on using joins may seem superfluous, but it is motivated by the cost of GC techniques. For instance, known symmetric-key GC schemes require at least one ciphertext (i.e., λ bits) of communication to implement the above conditional assignment. The switch system captures this in the sense that expressing the conditional assignment requires a single join. Each garbled join gate incurs λ bits of communication.

Switch systems also support XOR gates. These are standard fan-in-two gates, except that they act as *constraints*, so once any two connected wires become solved, the evaluator can solve for the third. Garbled XOR gates are communication free [KS08].

Finally, since switch systems require that the evaluator learns switch controls, we need a mechanism by which we reveal values. We consider a simple reveal gate which achieves this task. Again, for security one must prove a particular switch system has the property that all revealed values provide no useful information about the input – the switch system should be *oblivious* [Hea24].

We note that our handling of revealing bits is slightly different from that of [Hea24]. While we consider reveal gates, [Hea24] instead explicitly reveals the control bit of *each* switch. This is a small change, but it allows us to more precisely account for communication cost, since each revealed bit incurs one bit of communication. As it turns out, our merge construction requires *no* reveal gates, since all switch controls can be inferred from the merge's control string s. Hence the *only* communication in our merge comes from its join gates.

Scaling one-hot vectors via switch systems. One of the main known switch system tricks leverages *one-hot vectors*. A one-hot vector is a bit vector that contains exactly one entry equal to 1; other entries are 0. For example, $(q_0, q_1, q_2, q_3) = (0, 1, 0, 0)$ is a one-hot vector. In the context of a switch system, certain operations on one-hot vectors require very few join gates, and hence very little communication once garbled [HK21, Hea24]. In particular, an entire length-n one-hot vector can be *scaled* by a bit x at the cost of only a *single join*. This operation requires that the evaluator E learn the value of the one-hot vector in cleartext. When this can be safely achieved, garbling this operation requires only λ bits of communication, a *huge* improvement over the $O(n\lambda)$ bits that would be more typically consumed. Our merge construction extensively leverages the ability to efficiently multiply one-hot vectors with scalars.

We review [Hea24]'s switch system construction for scaling a one-hot vector. We consider the example problem of scaling vector $(q_0, q_1, q_2, q_3) = (0, 1, 0, 0)$ by x, yielding a scaled vector r: Each q_i is used to

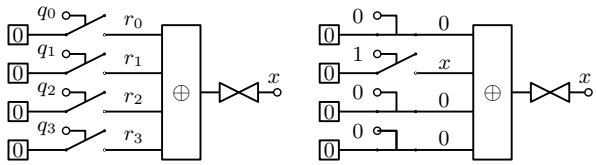


Figure 2: Scaling.

control a switch. Recall that switches ($_/_$) propagate their input if and only if their control is 0, depicted by the switch "snapping closed". Because q_0, q_2, q_3 are each zero, their respective switches close, which sets r_0, r_2, and r_3 to 0. The system then includes XORs that compute the sum of bits r_i. Finally, a single join gate insists that this sum is equal to x. Recall join gates ($\bowtie$) enforce that their two inputs carry the same value. Since r_0, r_2, and r_3 each hold 0, it follows that r_1 must be equal to x, yielding $(r_0, r_1, r_2, r_3) = (0, x, 0, 0)$ – the scaled one-hot vector.

Formally, the one-hot scaling can be expressed by the following switch system:

$$r_i := 0 \Leftarrow q_i \stackrel{?}{=} 0 \qquad \text{and} \qquad \bigoplus_i r_i = x$$

Since this system includes a single join, its garbling has size λ bits; again, this is a large improvement over a corresponding Boolean circuit which would incur cost $O(n\lambda)$ bits.

Garbled switch systems. We sketch how switch systems are garbled, using the above one-hot scaling operation as a motivating example. For rigorous discussion, see Section 4 and Appendix E.

To garble a switch system, the garbler assigns to each wire two ciphertexts, according to the standard free-XOR technique [KS08]. For example, in the one-hot scaling system, G assigns wire q_0 the ciphertexts $\{K_{q_0}, K_{q_0} \oplus \Delta\}$. Here, $K_{q_0} \in \{0,1\}^\lambda$ is a wire-specific key, and $\Delta \in \{0,1\}^\lambda$ is a global correlation. Recall that in the scaling system, wire r_0 is the output of a switch controlled by q_0. To garble this switch, G *derives* a key K_{r_0} from K_{q_0}:

$$0^\lambda \oplus H(K_{q_0}) \;=\; K_{r_0}$$

H can be understood as a random oracle. The same idea is applied for other wires q_i and r_i. Therefore, once the evaluator E obtains K_{q_0}, and once she learns that $q_0 = 0$, she can derive the corresponding key K_{r_0}, simply by computing $H(K_{q_0})$ herself. In general, consider a switch gate with inputs x, y and a control wire *ctrl*, whose ciphertexts are $(K_x, K_x \oplus \Delta)$, $(K_y, K_y \oplus \Delta)$, and $(K_{ctrl}, K_{ctrl} \oplus \Delta)$. The garbler G prepares the garbled switch by enforcing the following:

$$K_y \;=\; H(K_{ctrl}) \oplus K_x$$

When the encoding of x, i.e., $K_x \oplus x\Delta$, is available, the evaluator E can derive the encoding of y under the condition $ctrl = 0$ and $x = y$ as follows:

$$K_y \oplus y\Delta \;=\; K_x \oplus x\Delta \oplus H(K_{ctrl}).$$

To garble a join gate, the garbler G sends the difference $K_x \oplus K_y$ between the ciphertexts of the two wires x and y. Thus, when the evaluator E holds $K_x \oplus x\Delta$, it can obtain $K_y \oplus y\Delta$ by simply adding the difference to $K_x \oplus x\Delta$.

The garbled one-hot scaling switch system (showing the zero-valued keys) is as follows: Suppose q_0, q_2, q_3

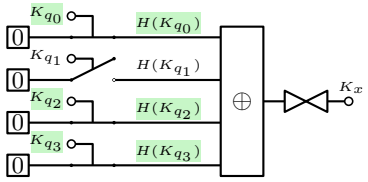


Figure 3: Scaling, showing the zero-keys of each wire.

are all 0, so that the evaluator E obtains $H(K_{q_0}), H(K_{q_2}), H(K_{q_3})$. Through the join gate, we then have

$$H(K_{q_0}) \oplus H(K_{q_2}) \oplus H(K_{q_3}) \oplus ??? \;=\; D \oplus K_x \oplus x\Delta,$$

where D is the difference of the ciphertexts of the join gate wires in the switch system $D = K_x \oplus \bigoplus_{i=0}^{3} H(K_{q_i})$. The solution is exactly $H(K_{q_1}) \oplus x\Delta$! Therefore E can obtain the scaled one-hot vector using only a single join gate by solving the constraint.

2.2 Garbled Merge as a Finite State Automaton

Recall that our main technical observation is to encode the partially-private merge of two arrays in the execution of a *finite state automaton* (FSA). As we will see in detail soon, state transitions in this FSA correspond to reading particular elements from either x^0 or x^1. From here, we observe that the execution of this FSA can be encoded by a particular switch system C, made efficient by carefully leveraging one-hot scaling operations. Because of this care, most gates in C are "free", in the sense that, when garbled, their garbling requires no communication. Indeed, C has only $O(nw)$ join gates, and these are C's only gates that incur communication.

We first demonstrate how to encode a merge in the execution of an FSA, and then how to encode the execution of that FSA as switch-system-based constraints.

Our FSA The stable merge of x^0 and x^1 can be described by a language accepted by a loop-free finite-state automaton (FSA). Let $m = |x^0|$ and $n = |x^1|$. Suppose that x^0, x^1 are strings over an alphabet Σ. We define an FSA $\mathcal{A} = (\mathcal{S}, \Sigma, \delta, S_{init}, S_{final})$ with state set $\mathcal{S} = \{q_{i,j} \mid 0 \le i \le m,\ 0 \le j \le n\}$, initial state $S_{init} = q_{0,0}$, and accepting state set $\mathcal{S}_{final} = \{q_{m,n}\}$. Transitions advance along a $(m{+}1) \times (n{+}1)$ grid:

$$\delta(q_{i,j},\, x_i^0) = q_{i+1,j} \quad \text{for } i < m, \qquad \delta(q_{i,j},\, x_j^1) = q_{i,j+1} \quad \text{for } j < n.$$

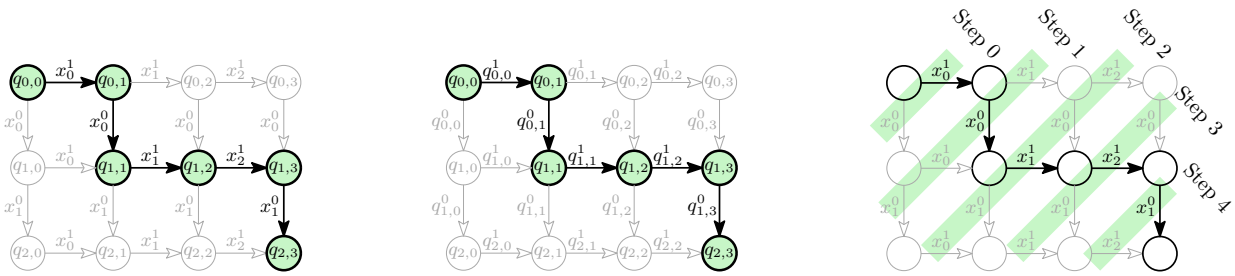
Intuitively, state $q_{i,j}$ denotes that i symbols have so far been taken from x^0, and j from x^1. A string $y \in \Sigma^{m+n}$ is accepted iff there is a path, or execution trace, from $q_{0,0}$ to $q_{m,n}$ whose edge labels, read in order, equal y. In the running example, the accepted trace $y = x_0^1 x_0^0 x_1^1 x_2^1 x_1^0$ corresponds to the following moves:

$$q_{0,0} \xrightarrow{x_0^1} q_{0,1} \xrightarrow{x_0^0} q_{1,1} \xrightarrow{x_1^1} q_{1,2} \xrightarrow{x_2^1} q_{1,3} \xrightarrow{x_1^0} q_{2,3}$$

See also Figure 4a.

Notice that each of $\mathcal{A}$'s feasible transitions is uniquely identified by a triple (b, i, j), where $b \in \{0, 1\}$ indicates which array is being read, and (i, j) indicates the current state. We assign to each such transition a binary indicator value $q_{i,j}^b$; see Figure 4b. These binary indicators specify which edges of $\mathcal{A}$ are *active* (resp. inactive), and they will be crucial to encoding $\mathcal{A}$ as a switch system.

With indicators $q_{i,j}^b$ defined, an accepting trace can now be uniquely characterized by computing products of the grid configuration $\{q_{i,j}^b\}$ with corresponding edge labels, which determines the sequence of outputs.



(a) (b) (c)

Figure 4: Three views of the merge process.

For example, the first character of the accepted string y can be determined:

$$y_0 \;=\; q^0_{0,0} \cdot x^0_0 \;\oplus\; q^1_{0,0} \cdot x^1_0 \;=\; \left(\bigoplus_{i+j=0} q^0_{i,j} \cdot x^0_j \right) \;\oplus\; \left(\bigoplus_{i+j=0} q^1_{i,j} \cdot x^1_i \right)$$

This holds because, after the first step, the only reachable states are $q_{0,1}$ and $q_{1,0}$, reached via x^1_0 and x^0_0, respectively. The inclusion of a particular transition at this step is determined by the binary indicators $q^1_{0,0}$ and $q^0_{0,0}$.

More generally, each output symbol y_k can be determined by computing a sum of products along *diagonals* of the FSA's grid:

$$y_k \;=\; \left(\bigoplus_{i+j=k} q^0_{i,j} \cdot x^0_j \right) \;\oplus\; \left(\bigoplus_{i+j=k} q^1_{i,j} \cdot x^1_i \right) \tag{1}$$

See also Figure 4c. Thus, to encode the FSA's execution, it suffices to perform the follow computations:

- From the control bitstring s, compute indicators for each state $q_{i,j}$ and each transition $q^b_{i,j}$.
- From the transition indicators and edge labels (i.e., data elements in x^0, x^1), compute each value y_k.

These computations can be efficiently expressed as a switch system.

Merge as a switch system. As a first attempt, suppose we somehow compute all indicators $q_{i,j}, q^b_{i,j}$, and now we would like to compute each output y_i. To do so, we can naïvely encode Equation (1) with Boolean gates, but the cost will be too high. Indeed, these equations involve $O(nmw)$ Boolean multiplications (ANDs), which would ultimately require $O(nmw\lambda)$ bits of communication once garbled. Our goal is to instead consume only $O((n+m)w\lambda)$ bits of communication.

The key to achieving this lower communication is to (1) recall that one-hot vectors support an efficient garbled scaling vector and (2) observe that any accepting execution of the FSA $\mathcal{A}$ is laden with one-hot vectors. Indeed, by examining Figure 4 we can observe numerous one-hot vectors:

- Along each k-th diagonal, the state indicator variables $\{q_{i,j} \mid i+j=k\}$ form a one-hot vector.
- Similarly, along each k-th diagonal, the transition indicator variables $\{q^b_{i,j} \mid i+j=k\}$ form a one-hot vector.
- Each i-th horizontal collection (row) of transition indicators $\{q^0_{i,j}\}$ forms a one-hot vector. Each such i-th row specifies *when* the i-th element x^0_i should be read (i.e., for which j should y_j be equal to x^0_i).
- Similarly, each i-th vertical collection (column) of transition indicators form a one-hot vector, each corresponding to when element x^1_i should be read.

These one-hot vectors emerge because $\mathcal{A}$'s accepting executions proceed monotonically down and to the right through its grid.

We leverage these one-hot vectors to express desired constraints over the FSA far more communication-efficiently:

Transition indicators. The k-th diagonal of transition indicators can be efficiently computed from the k-th diagonal of state indicators. Indeed, these two diagonals are related by a one-hot scaling operation, and some XORs. For instance, we can examine relationships on diagonal $k = 1$:

$$\begin{aligned}[q^1_{0,1},\ q^1_{1,0}] &:= s_1 \cdot [q_{0,1},\ q_{1,0}] \\ [q^0_{0,1},\ q^0_{1,0}] &:= [q_{0,1},\ q_{1,0}] \oplus [q^1_{0,1},\ q^1_{1,0}]\end{aligned}$$

This works because, when $s_1 = 1$, exactly one of the edges $q^1_{0,1}$ or $q^1_{1,0}$ is taken in the transition, so the vector $[q^1_{0,1}, q^1_{1,0}]$ contains exactly one non-zero entry. When $s_1 = 0$, neither of these edges is used in the transition, so $[q^0_{0,1}, q^0_{1,0}]$ becomes the all-zero vector, and the constraint remains valid. In this latter case, one downward transition $q^0_{0,1}$ or $q^0_{1,0}$ must be taken, and the above XOR constraints ensure this holds. In Section 5, we extend this to every $k = 1, \ldots, n + m$, where each case requires only one join gate. Thus, this part of the switch system uses only $n + m$ join gates.

State indicators. The k-th diagonal of state indicators can be straightforwardly computed from the $(k-1)$-th diagonal of transition indicators. Indeed, we simply define each state as the XOR of its incoming transitions. These constraints require no join gates.

Read values. Values read from x^0 and x^1 can be constrained by transition indicators. For example, element x^0_0 is read at exactly out of four possible points in time, each corresponding to a distinct transition $q^0_{0,j}$. Since only one such transition can hold one, these transitions form a one-hot vector, which we scale by the element x^0_0 at cost w join gates (one join for each of x^0_0's bits). We define $read^0_{0,j}$ as a variable that stores x^0_0 if and only if x^0_0 is read on transition $q^0_{0,j}$. These variables are constrained as follows:

$$[read^0_{0,0}, read^0_{0,1}, read^0_{0,2}, read^0_{0,3}] := x^0_0 \cdot [q^0_{0,0}, q^0_{0,1}, q^0_{0,2}, q^0_{0,3}],$$

where $[q^0_{0,0}, q^0_{0,1}, q^0_{0,2}, q^0_{0,3}]$ is a one-hot vector. In Section 5, we extend this constraint to every x^0_i for $i = 1, \ldots, m$ and x^1_j for $j = 1, \ldots, n$. With these $read^b_{i,j}$ variables, we can then simply define each merge output y_k as follows:

$$y_k := \left(\bigoplus_{i+j=k} read^0_{i,j} \right) \oplus \left(\bigoplus_{i+j=k} read^1_{i,j} \right)$$

This constraint uses only free XOR operations. Altogether, this construction requires only $w(n + m)$ join gates, w for each element in x^0 and x^1.

Putting everything together, the entire switch system consists of only $(w + 1)(n + m)$ joins, and hence its garbling has size $(w + 1)(n + m)\lambda$ bits. This is possible because our approach bridges efficient switch-system-based garbling techniques and the garbled merge problem by means of a carefully chosen FSA.

Some important technical details remain. For instance, while we have constrained all variables in the FSA, we have not carefully shown here that an evaluator can indeed *solve* that system. We present such explanations, along with formal presentation of all constraints, in Section 5.

3 Preliminaries

3.1 Basic Notation

- λ denotes a security parameter, which can be understood as the bit-length of encryption keys (e.g. 128).
- $X \stackrel{c}{=} Y$ denotes ensembles X and Y are computationally indistinguishable (in λ).
- $[n]$ denotes the natural number $\{0, ..., n - 1\}$.

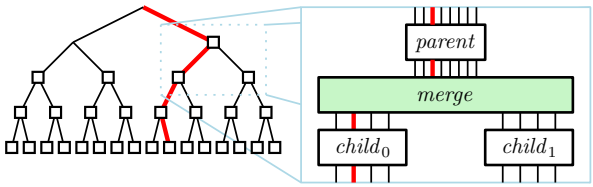


Figure 5: Structure of [HKO23]'s GRAM, which we upgrade by installing our new merge. The GRAM implements the binary tree structure of Circuit ORAM [WCS15]. Circuit ORAM stores each memory element along the path towards a particular leaf; accessing an element requires traversing its path, as well as obliviously rearranging elements on paths. Merge components allow for runtime routing between parent nodes and child nodes, allowing them to send to one another. Our improved merge asymptotically reduces the size of the GRAM.

- $x := y$ denotes that x is equal to y by definition. This form of equality is crucial in the switch system model, since in a switch system the constraint $x := y$ is communication free, whereas the constraint $x = y$ requires a join gate, which incurs λ bits of communication once garbled.
- H is a circular-correlation robust hash function; see Section 3.3.
- Generally speaking, we consider the merge of two arrays: x^0, which has length n, and x^1, which has length m. Array elements are of length w.

3.2 Garbling Schemes

Our technique can be formalized by means of a *garbling scheme* [BHR12]. Roughly, a garbling scheme is a tuple of procedures that specify ow to garble a program and how to evaluate that garbled program. The crucial properties are *correctness*, which states that the garbled program outputs the same value as the cleartext program (with overwhelming probability) and *privacy*, which states that the garbled program can be simulated, given only the program and its output (and not the input). Appendix A provides a full definition.

3.3 Cryptographic Assumption

Our garbling techniques are secure assuming the existence of a hash function satisfying a concrete property called *circular correlation robustness* (CCR) [CKKZ12, ZRE15]. It is typical to instantiate a CCR hash via fixed-key AES, and this instantiation is relatively well studied; see e.g. [BHKR13, GKWY20]. Appendix B provides a formal definition.

3.4 Tri-State GRAM; How Our Merge Plugs In

Our main contribution is an improved GRAM construction, based on our garbled merge primitive. Specifically, we build upon the Tri-State GRAM of [HKO23]. We replace one of its core components—circuit-based stacks—with our more efficient garbled merge.

While the [HKO23] GRAM is formalized as a so-called *tri-state circuit* (TSC), not as a switch system, the switch system model is formally more expressive that the TSC model; [Hea24] provides a proof. Every (total) TSC is automatically and directly implementable as a switch system. Indeed, switch systems are a natural generalization of TSCs. Hence, the [HKO23] GRAM can be understood as a switch system, allowing clean interoperability of our techniques.

This section reviews the Tri-State GRAM, with a focus on its stacks and their cost. We then explain why the circuit-based stacks can be correctly replaced by our garbled merge. Finally, we analyze the cost of our garbled merge in the new GRAM construction.

Stacks in Tri-State GRAM As shown in Figure 5, Tri-State GRAM [HKO23] is structured as a binary tree. Each internal node maintains two circuit-based stacks [ZE13], one for each child. These stacks are used to solve the runtime routing problem, allowing parent nodes and child nodes to dynamically communicate. Each stack supports a *conditional pop (cpop)* operation: given a control bit p, the stack returns its top element if $p = 1$, or an all-zero string if $p = 0$. For each access, the parent uses the data's traversal path in the tree to determine which stack receives a control bit of 1, while the other receives 0. The children manage the stack contents and return values accordingly. For a stack of size t with word size w, the [ZE13] implementation incurs $O(w \cdot t \lg t)$ joins.

Replacing Stacks with Garbled Merge Each pair of stacks in Tri-State GRAM can be replaced by one instance of our garbled merge primitive. Consider two stacks of size t that each supports $2t$ cpop operations (of which t have control bit 0 and t have control bit 1). Let x^0 be the t w-bit elements of one stack, let x^1 be the t w-bit elements of the other stack, and let $s \in \{0,1\}^{2t}$ with Hamming weight n be the sequence of control bits. Suppose we call cpop $2t$ times on each stack (the x^0 stack using negated control string $\neg s$, the x^1 stack using s). If we element wise XOR the two resulting arrays of length $2t$, we obtain precisely the merge of x^0 and x^1 as prescribed by control bits s. By Theorem 2, to replace two stacks of size t, our garbled merge approach incurs $(w+1) \cdot (2t)$ (or $O(w \cdot t)$) joins. This is a factor-$\lg t$ improvement over the stack construction in Tri-State GRAM.

While our garbled merge requires that the evaluator learn the control string s, revealing this value does not compromise obliviousness of GRAM. Indeed, garbled merge only leaks the control string s, which corresponds precisely to the leakage of cpop operations in the original Tri-State GRAM. Each bit in s indicates which child node is active and thus reveals the traversal path down the tree. However, this path information is permitted to leak in the underlying Circuit ORAM construction [WCS15]. Even with the path revealed, Circuit ORAM ensures that the actual memory access pattern remains hidden. Therefore, the obliviousness of the GRAM construction is preserved.

Stacks support online behavior by outputting one element at a time during cpop operations. We emphasize that our garbled merge primitive preserves this property. As previously discussed, our switch-system-based FSA is online: it allows merging to proceed as control bits arrive, and partial outputs can be read immediately. Thus, garbled merge supports online output in a manner similar to that of stacks.

Cost of Garbled Merge In the Tri-State GRAM [HKO23], the RAM program is represented as a binary tree. Consider a word-RAM program with word size w that halts in n steps. At level i of the tree, there are 2^i nodes. A naive approach would allocate a stack of size $O(n/2^i)$ for each node to support $O(n/2^i)$ accesses.

However, Tri-State GRAM adopts a technique from NanoGRAM [PLS23] to reduce stack sizes using a balls-and-bins argument. Let $p = \lg n \cdot \omega(1)$ be a super-logarithmic parameter. Since Circuit ORAM selects access paths randomly, with overwhelming probability, in any batch of $2^i \cdot p$ accesses, no node at level i is accessed more than $2 \cdot p$ times. Thus, each node can use stacks of size $2 \cdot p$, *resetting* them every $2^i \cdot p$ accesses. This results in $\Theta(n/(2^i \cdot p))$ stacks per node at level i, each of size $2 \cdot p$. [3]

In Tri-State GRAM, using stacks of size $\lg n \cdot \omega(1)$ incurs an overhead of $O(\lg \lg n)$, as their circuit-based stack construction introduces overhead logarithmic in the stack size. In contrast, our garbled merge construction allows choosing *any* super-logarithmic p without incurring asymptotic overhead. Increasing p further will decrease concrete communication (constant overhead in gates is needed to reset stacks), but increase computation.

It is worth noting that, in the Tri-State GRAM setting, the size of the stacks is $O(p) = O(\lg n) \cdot \omega(1)$, which means we only need to handle relatively small stacks, typically on the order of only hundreds of elements to achieve concretely low failure probability.

By replacing the stacks with our garbled merge construction (as described above) and applying Theorem 2, each such stack on level i incurs $O(w \cdot p)$ join gates. Hence, the total number of joins across all levels is:

$$\sum_{i=0}^{\lg n} 2^i \cdot \Theta\left(\frac{n}{2^i \cdot p}\right) \cdot O(w \cdot p) = O(w \cdot n \cdot \lg n)$$

[3] Technically, those levels i close to the leaves where each node has fewer than p descendants have 2^i stacks of size $O(n/2^i)$. These are cheaper and do not impact overall asymptotics.

The resulting communication cost is therefore $O(w \cdot n \cdot \lg n \cdot \lambda)$, achieving a $\lg \lg n$-factor improvement over [HKO23]. This optimization is central to reducing the cost of our GRAM construction.

In Tri-State GRAM, the total communication is $O(n \cdot (w + \lg^2 n) \cdot \lg n \cdot \lg \lg n \cdot \lambda)$, dominated by stack-based merging, which contributes $O(w \cdot n \cdot \lg n \cdot \lg \lg n \cdot \lambda)$ from the main GRAM tree and $O(n \cdot \lg^3 n \cdot \lg \lg n \cdot \lambda)$ from recursive position maps, which are themselves GRAM trees. By reducing the merging cost by a $\lg \lg n$ factor in both the main and recursive trees, we achieve a corresponding $\lg \lg n$-factor reduction in the overall communication cost.

4 Garbled Switch System Review

This section reviews the formal definition of switch systems and the manner by which they are garbled. Technically, we introduce a small change to [Hea24]'s original formulation. Namely, we formalize a new *reveal* gate, rather than having *each* switch reveal its control bit. (For simplicity, our formalism also does not include special-purpose gate types needed for arithmetic operations.) Because of this change, Appendix E provides the complete garbling scheme specification and proof of security.

Definition 1 (Switch System)**.** *A **switch system** S specifies a system of constraints on wires holding bits. The system is defined in terms of* gates*. Non-input wires in the system are initially **not set** (i.e., have no value), and as the system runs, wires become **set** according to the rules of each gate. The types of gates are as follows:*

- *A **reveal gate** takes as input a wire x. When x becomes set, the system marks that wire as **revealed**. When a value can be computed (in polynomial time) from values on revealed wires, we say that value is **deducible**.*
- *A **switch** takes as input a* control wire *ctrl and* data wire *x. It outputs data wire y. If ctrl is set and it is deducible that $ctrl = 0$, then the gate establishes constraint $x = y$. We denote a switch by writing $y := x \Leftarrow ctrl \stackrel{?}{=} 0$.*
- *A **join** takes input wires x, y and establishes constraint $x = y$. We denote a join by simply writing $x = y$.*
- *An **XOR** takes input wires x, y, and it outputs wire z. We denote an XOR by writing $z := x \oplus y$.*

*A switch system S has n **input wires** and m **output wires**. We denote by $S(x)$ the values on outputs after running with input $x \in \{0,1\}^n$. S is considered **valid** if for every input x, each wire becomes set, and the setting of each wire is unique (i.e. determined by x). From here, all considered switch systems will be valid.*

Reveal Gates; Obliviousness At a high level, a switch is garbled by invoking a CCR hash function H. H is used to encrypt the difference between data wire labels according to the control wire's zero label. When the control holds zero, the GC evaluator E hashes her corresponding control label to decrypt the difference between data labels, allowing her to translate labels on data wires x and y. If the control holds 1, E instead does nothing, as in this case the switch imposes no constraint on x and y.

Since E's behavior is conditioned on the control wire's semantic value, E must know that value in cleartext. Hence, we must sometimes reveal values to E such that she can *deduce* whether each switch is open or if it is closed. This is the purpose of *reveal gates* – these gates reveal to E cleartext values on specified wires. Each reveal gate incurs 1 bit in terms of the size of the garbled program. This presentation is slightly different than that of [Hea24], where *each* switch gate control wire was revealed, possibly leading to redundant bits being sent. Our inclusion of a reveal gate makes precise how many extra bits need be included in the garbled program.

Our end goal is to construct privacy-preserving computations, so ultimately we must construct switch systems where the revealed bits do not disclose *useful* information. This motivates the definition of *oblivious* switch systems; the following is adjusted from [Hea24]:

Definition 2 (Oblivious Switch System)**.** *A **randomized switch system** is a pair consisting of a switch system S and a distribution on bitstrings D. The execution of a randomized switch system on input x is defined by sampling $r \in_\$ D$, then running S on x and r:*

$$(S, D)(x) := \{\ S(x; r) \quad \textit{where} \quad r \in_\$ D\ \}$$

Let $\mathcal{L}(S, x)$ denote the set of revealed wire values in S (each labeled by its wire ID) obtained when executing $S(x)$. We say a family of switch systems $\{S_i, D_i\}_i$ computes a function f if the following holds with overwhelming probability:

$$\Pr\left[(S_\lambda, D_\lambda)(x) = f(x)\right] > 1 - negl(\lambda)$$

*A family $\{S_i, D_i\}_i$ that computes f is considered **oblivious** if there exists a simulator $\mathcal{S}_{reveal}$ such that the following ensembles are statistically close in λ:*

$$\mathcal{S}_{reveal}(1^\lambda) \stackrel{s}{=} \{\ \mathcal{L}(S_\lambda, (x; r)) \quad \mid \quad r \in_\$ D_\lambda\ \}$$

For completeness, Appendix E provides a full garbling scheme (based on the scheme in [Hea24]), which establishes the following:

Theorem 1 (Garbling Scheme for Switch Systems)**.** *Assuming a circular correlation robust hash function H (Definition 6), there exists a correct and private garbling scheme $\mathbb{S}$ (Definition 5) for oblivious switch systems (Definition 1). Let (S_i, D_i) denote an oblivious switch system family. $\mathbb{S}$ has the following performance characteristics:*

- *Let n_{join} denote the number of joins in S_λ, n_{reveal} the number of reveals, and n_{out} the number of outputs. A garbling $\tilde{S}_\lambda$ of S_λ has size $n_{join} \cdot \lambda + n_{reveal} + n_{out}$ bits.*
- *Let n_{switch} denote the number of switches in S_λ. G issues n_{switch} calls to the hash function H. E issues at most n_{switch} calls to H.*

Gate costs Garbled switch system gates incur the following costs:

	G Calls to H	E Calls to H	Communication (bits)
XOR	0	0	0
Switch	1	0 or 1	0
Join	0	0	λ
Reveal	0	0	1

Table 2: Per-gate costs in the garbled switch system.

5 Garbled Merge

This section formalizes our switch system (Section 5.1) and carefully argues its correctness and size. Section 5.2 then explains an optimization to our system that becomes available when the merge's control string s is random. Finally, Section 5.3 formalizes our GRAM which, as discussed in Section 3.4, is obtained by simply plugging our merge into the existing Tri-State GRAM of [HKO23].

5.1 Merge as a Switch System

Recall that our goal is to merge two arrays x^0 and x^1 as determined by a string of control bits s. We formalize online stable merge:

Definition 3 (Online Stable Merge)**.** *Let x^0 (resp. x^1) be an n-element (resp. m-element) array of w-bit entries. An **online stable merge** of x^0 and x^1 is an online algorithm that maintains two integers i and j, each initially set to zero. At each step k, the algorithm accepts a bit $s_k \in \{0, 1\}$. We refer to the sequence of all such bits $s \in \{0, 1\}^{n+m}$ as the* control string*. We assume s has Hamming weight m. Upon receiving s_k, the algorithm proceeds as follows:*

- *If $s_k = 0$, output $y_k := x^0_i$, then increment i.*
- *If $s_k = 1$, output $y_k := x^1_j$, then increment j.*

After $n + m$ total calls, the algorithm halts. Thus the algorithm implicitly defines a length-$(n + m)$ array y.

The main goal of this section is to construct a switch system that computes the above:

Theorem 2 (Switch System Merge)**.** *There exists a switch system C that computes the online stable merge of arrays x^0 and x^1, given that the control string s is deducible. C has $O(wmn)$ total gates, including $(w + 1) \cdot (m + n)$ join gates and no reveal gates.*

The proof of this theorem is by construction of C, formalized in Equations (2) to (7). Correctness and size of this switch system is argued throughout this section.

Following Section 2, our FSA for merging length-n array x^0 and length-m array x^1 consists of a $(n+1) \times (m + 1)$ two-dimensional[4] grid of states, where each state $q_{i,j}$ indicates that i elements from x^0 and j from x^1 have been read.

For now, set aside the problem of reading elements from x^0, x^1, and instead focus on the simpler task of encoding the execution of such an FSA as a switch system; once we achieve this, it will be relatively easy to read elements from x^0, x^1.

Encoding the FSA as constraints. Recall from Section 2 that we described FSA indicator variables $q_{i,j}, q^b_{i,j}$. We specify switch-system-based constraints on these indicator variables. To do so, we encode each such variable by a switch system wire. Those indicators that are *active* in a particular execution are encoded by logical 1, and those that are inactive are encoded by logical 0.

Our goal is to constrain each state indicator $q_{i,j}$, each downward indicator $q^0_{i,j}$, and each rightward indicator $q^1_{i,j}$.

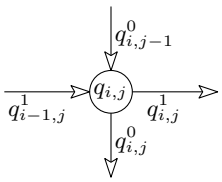


Figure 6: State indicator $q_{i,j}$ with its adjacent downward indicators ($q^0_{i,j-1}$, $q^0_{i,j}$) and rightward indicators ($q^1_{i-1,j}$, $q^1_{i,j}$).

For simplicity of notation in our constraints, we introduce the following notion of *valid* indicator. In short, a state/transition indicator with coordinate i, j is valid if it refers to a place in the FSA's two-dimensional grid; an example of an invalid state is one with a negative i coordinate:

Definition 4 (Valid FSA States/Transitions)**.** *State indicator $q_{i,j}$ is valid iff $i \in [n + 1]$ and $j \in [m + 1]$. Rightward transition indicator $q^1_{i,j}$ is valid iff $i \in [n + 1]$ and $j \in [m]$. Downward transition indicator $q^0_{i,j}$ is valid iff $i \in [n]$ and $j \in [m + 1]$.*

Our constraints will implicitly refer only to valid indicators, and summations will implicitly range only over valid indicators.

Constraining State Indicators. Each FSA state indicator $q_{i,j}$ is constrained by its incoming transition indicators. Namely, each state is active if either of its incoming transitions is active. Formally, we define

[4]Our FSA can be naturally generalized to a d-dimensional grid, allowing to garble a merge on d arrays. However, the computation overhead of this approach would be exponential in d. A computationally cheaper approach is to use our merge on two arrays recursively, so we do not pursue the d-dimensional approach further.

each state indicator $q_{i,j}$ as follows:

$$q_{i,j} := \begin{cases} 1 & \text{if } i = 0 \text{ and } j = 0 \\ q^0_{i-1,0} & \text{if } i > 0 \text{ and } j = 0 \\ q^1_{0,j-1} & \text{if } i = 0 \text{ and } j > 0 \\ q^0_{i-1,j} \oplus q^1_{i,j-1} & \text{otherwise} \end{cases} \tag{2}$$

Constraints on states do not require any joins, and so they are "free" to garble.

Constraining Transition Indicators. Recall that at the k-th step of execution, the FSA is in some active state $q_{i,j}$ where $i + j = k$. The k-th active transition should be transition $q^{s_k}_{i,j}$, where s_k is the k-th control bit.

The possible active transitions at step k lie along the k-th diagonal of the grid. For each such k-th diagonal, our goal is two-fold:

- We wish to force the single active transition on the k-th diagonal to hold logical 1.
- We wish to force *all other transitions* on the k-th diagonal to hold logical 0.

We start constraining transition indicators as follows:

$$q^0_{i,j} := \begin{cases} q_{i,m} & \text{if } j = m \\ q_{i,j} \oplus q^1_{i,j} & \text{else} \end{cases} \quad q^1_{i,j} := \begin{cases} q_{n,j} & \text{if } i = n \\ 0 \Leftarrow q_{i,j} \stackrel{?}{=} 0 & \text{else} \end{cases} \tag{3}$$

In both definitions, the first case is special: if a state lies on the right (resp. bottom) of the grid, then its outgoing downward transition (resp. rightward transition) is active iff the state is active. More interesting are the general cases. The rightward-transition general case uses a single switch gate to insist that if a state $q_{i,j}$ is inactive, then so is its outgoing rightward transition $q^1_{i,j}$. In other words, an active transition can only come from an active state.

The downward transition general case states that the transition $q^0_{i,j}$ is the difference between state $q_{i,j}$ and its rightward transition $q^1_{i,j}$. Note that if $q_{i,j}$ is inactive (i.e., holds logical zero), then so is its rightward transition $q^1_{i,j}$, so the downward-transition general case will also set $q^0_{i,j}$ as inactive. In contrast, if $q_{i,j}$ is *active*, the constraints ensure that $q^0_{i,j}$ is *different* than $q^1_{i,j}$: if the active state's rightward transition is active, then its downward transition is inactive, and vice versa.

We have now constrained all transitions coming from inactive states, and we have constrained the single *downward* transition from the active state, but the *single rightward transition from the active state* is not yet constrained. For each $k \in [n + m]$, we encode the following constraint with a single join:

$$\bigoplus_i q^1_{i,k-i} = s_k \tag{4}$$

The above constraint *scales* the one-hot vector $(q^1_{0,k}, \ldots, q^1_{k,0})$ by s_k (see Section 2's discussion of one-hot vectors), setting the rightward transition from the active state to s_k, and by Equation (3), the corresponding downward transition to $1 - s_k$. This ensures that the single active transition on the diagonal is constrained to 1.

Correctness – How Indicators are Propagated. We explain how the above constraints correctly implement FSA execution. We use our running example to demonstrate a single step of FSA execution, and explain how it generalizes. Suppose that we have so far taken one step in the FSA, moving from state $q_{0,0}$ to state $q_{0,1}$. We illustrate how E uses control wire $s_1 = 0$ to next move from $q_{0,1}$ to $q_{1,1}$.

Consider those FSA states above and/or left of active state $q_{0,1}$. Recall that our merge system must implement an online algorithm, where the control bits s_k arrive one by one. Thus, we need to demonstrate that as those bits arrive, the solution for switch system indicators will correctly evolve. Formally, correctness of our switch system proceeds by the following inductive invariant:

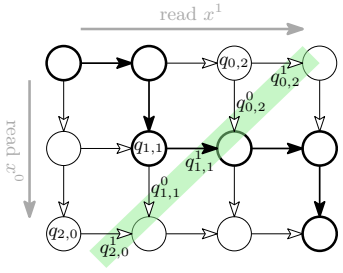


Figure 7: One of our FSA's diagonals.

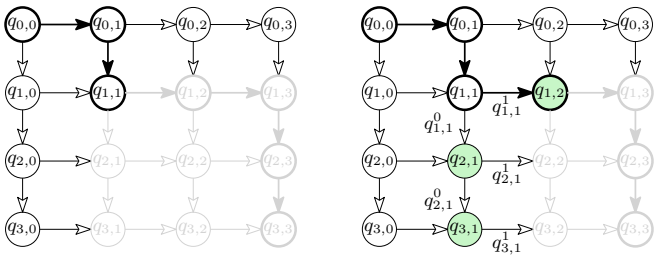


Figure 8: A step of our FSA's execution.

Invariant 1. *We define* ***step*** k *of FSA evaluation to mean that* k *control bits* $s_0, ..., s_{k-1}$ *have been input to the online algorithm so far. Let* $q_{i,j}$ *where* $i + j = k$ *denote step* k*'s active state indicator (i.e.,* $q_{i,j} = 1$*). All indicators above and/or to the left of* $q_{i,j}$ *can be solved. Namely, the bits* $s_0, ..., s_{k-1}$ *combined with Equations* (2) *to* (4) *determine all following indicators, where* $i' < i$ *and* $j' < j$*:*

$$q_{i,j} \qquad q_{i',j'} \qquad q^0_{i',j'} \qquad q^1_{i',j'}$$

By induction, suppose Invariant 1 holds at step $k = i + j$. We must show that the arrival of bit s_k is sufficient to show Invariant 1 holds at step $k + 1$.

For example, Figure 8 shows a particular case where $i = 1, j = 1$. Encodings of states/transitions depicted in black in the left-hand diagram are solved. Elements depicted in gray are not yet solved.

For simplicity, our formal discussion assumes that i, j is somewhere in the middle of the FSA grid. That is, $0 < i < n$ and $0 < j < m$. The special cases are trivial simplifications of the following discussion.

Assume that the next control bit s_k arrives. First, we can solve for the two outgoing transitions $q^0_{i,j}, q^1_{i,j}$. Equation (4) enforces that all rightward transitions on each k-th diagonal must sum to s_k. Notice that Invariant 1 implies that all rightward transitions $q^1_{i',j'}$ where $i' + j' = k$ and $(i', j') \neq (i, j)$ are already solved: these must occur above or to the left of our active state $q_{i,j}$. Moreover, they must all be zero, since they come from inactive states $q_{i',j'}$. Thus, Equation (4) forces $q^1_{i,j} = s_k$. Next, we can apply Equation (3) to solve for the downward transition: $q^0_{i,j} := q_{i,j} \oplus q^1_{i,j}$. Notice that because $q_{i,j} = 1$, it must be that $q^{s_k}_{i,j} = 1$.

We can now apply Equation (2) to solve for the two successor states of $q_{i,j}$. Namely, we can solve for $q_{i+1,j}$ and $q_{i,j+1}$ by XORing their respective incoming transitions.

From here, behavior branches, depending on the value of s_k. If $s_k = 1$, the FSA transitions rightwards. To restore Invariant 1, we must compute all indicators *left* of the new active state $q_{i,j+1}$. This is possible because we have already deduced that the *downward* state $q_{i+1,j}$ holds zero, so we use Equation (3) to solve for its outgoing transitions. Then, we use Equation (2) to solve for $q_{i+2,j}$, which must also be inactive, allowing us to use Equation (3) to solve for its outgoing transitions, and so on. Symmetrically, if $s_k = 0$, the FSA transitions downward, and we repeatedly use Equations (2) and (3) to solve for all indicators above the new active state.

We have now restored Invariant 1: all indicators above and to the left of the new active state are determined. Notice that the above discussion clearly demonstrates the online nature of the merge.

Using the FSA to read. Now that we have specified constraints on the FSA's indicator variables, we can formalize constraints that read elements from the input arrays x^0, x^1.

We begin by recalling the semantics of an active transition: if $q^0_{i,j} = 1$, then x^0_i is the element read at step $k = i + j$. Since x^0_i is read only once, this implies $q^0_{i,k} = 0$ for all $k \neq j$. The same holds symmetrically for $q^1_{i,j}$.

We define variables $read^0_{i,j}$ and $read^1_{i,j}$ as follows:

$$read^0_{i,j} := 0^w \Leftarrow q^0_{i,j} \stackrel{?}{=} 0 \qquad read^1_{i,j} := 0^w \Leftarrow q^1_{i,j} \stackrel{?}{=} 0 \tag{5}$$

$$\bigoplus_j read^0_{i,j} = x^0_i \qquad \bigoplus_i read^1_{i,j} = x^1_j \tag{6}$$

Equations (5) and (6) scale the one-hot vector $(q^0_{i,0}, \ldots, q^0_{i,m})$ by x^0_i to produce $(read^0_{i,0}, \ldots, read^0_{i,m})$; symmetrically, we constrain $read^1_{i,j}$.

We can now constrain the k-th output element y_k of the merged array. At step k, exactly one transition on the k-th diagonal is active: there exists a unique i such that $q^{s_k}_{i,k-i} = 1$. And all other transitions are inactive; see Figure 7

If $s_k = 0$, then x^0_i is being read, and thus $y_k = x^0_i$. We have:

$$read^0_{i,k-i} = x^0_i, \quad read^0_{j,k-j} = 0 \text{ for } j \neq i, \quad read^1_{t,k-t} = 0 \text{ for all } t.$$

The case $s_k = 1$ is symmetric. Therefore, y_k can be computed using the following equation:

$$y_k := \bigoplus_i read^0_{i,k-i} \oplus read^1_{i,k-i} \tag{7}$$

Note that these constraints support merge's online nature. Indeed, Invariant 1 and Equation (5) together imply that all values $read^b_{i',j'}$ corresponding to *inactive* transition indicators $q^b_{i',j'}$ above and to the left of the active state $q_{i,j}$ must be equal to zero. This implies that as soon s_k arrives, one can (1) solve for $q^{s_k}_{i,j}$, then (2) use Equation (6) to solve $read^{s_k}_{i,j}$, and (3) use Equation (7) to obtain y_k.

Space Complexity. Recall that one of our main goals is to improve space complexity. The space complexity of the garbler (resp. evaluator) can be upper bounded by summing two quantities:

- The size of the garbled program.
- The number of wires that must be kept in memory to garble (resp. evaluate) the remaining wires, multiplied into λ. We wish to minimize this quantity without recomputing the encoding of any wire.

In Boolean GC, these two quantities are typically related by a constant factor, as the number of wires is proportional to the number of gates, and AND gates typically incur $O(\lambda)$ bits of communication. However, in our construction this is *not* the case. Our switch system has $O(nmw)$ wires, but its garbling has size only $O((n+m)w\lambda)$ bits.

Despite the fact that our construction has a large number of wires, it can be garbled (resp. evaluated) in only $O((n+m)w\lambda)$ bits of space. To show this, and since the garbling of our program fits within this asymptotic budget, it suffices to show that the garbler (resp. evaluator) needs only store $O((n+m)w)$ wire encodings.

Let us first discuss the evaluator's space. We can first examine the space needed to propagate indicator variables. Invariant 1 clearly implies that it suffices to keep track of the encodings of at most $O(n+m)$ indicators. Indeed, because the FSA has a grid structure, it suffices to keep the encoding of the single active state $q_{i,j}$ as well as those indicators exactly one step above and/or exactly one step to the left of $q_{i,j}$. There are only $O(n+m)$ such indicators.

The evaluator must also track values $read^b_{i,j}$ (Equations (5) to (7)). These are easy to handle in $O((n+m)w\lambda)$ bits of space. In particular, notice that Equations (6) and (7) involve XOR sums of variables $read^b_{i,j}$, and these are the *only* ways that these variables are used. Thus, rather than storing an encoding of each variable $read^b_{i,j}$ in local memory, the evaluator instead keeps track of $O(nm)$ accumulators (each corresponding to a w-bit word) corresponding to partial sums in those equations. As soon as the evaluator computes a new encoding $read^b_{i,j}$, she eagerly XORs into the appropriate accumulators, then discards the encoding.

The garbler's handling is easier than the evaluator's. In short, the garbler can proceed row by row through the FSA, using indicator encodings on row i to derive indicator encodings and *read* values on row $i+1$, as described by the switch system constraints. Once he has garbled row $i+1$, he discards the encoding of row i. To handle encodings of variables $read^b_{i,j}$, the garbler follows the evaluator's strategy and maintains accumulators rather than keeping all such variables in memory. Thus, he also runs in $O((n+m)w\lambda)$ space.

5.2 Randomized Merge

One problem of our merge construction is that it imposes high computational cost. Suppose that our two merged arrays x^0, x^1 are each of length n. Our construction presented so far requires that G and E each invoke a CCR hash function $O(w \cdot n^2)$ times, which is expensive. However, for particular use-cases – namely those where the control bits are randomly distributed – we can do far better, invoking the hash function only $O(w \cdot n \cdot \sqrt{n \cdot \lambda})$ times.

Suppose that the control bit string s is sampled uniformly, subject to the constraint that it has Hamming weight n. Note that the Tri-State GRAM's usage of merge has this property. By a simple observation about random walks, we can see that it is only negligibly likely that a random s will cause an FSA execution to deviate significantly from the main FSA's main diagonal. That is, it is unlikely the FSA will enter a state $q_{i,j}$ where $|i - j|$ is a large value.

In particular, an active path through the FSA is likely to deviate only a small distance $O(\sqrt{n \cdot \lambda})$ from the FSA's main diagonal. Because of this, we can greatly improve G's and E's compute by simply *ignoring* those states that are too far from the main diagonal, and our construction will still correctly implement merge with overwhelming probability. Appendix C expands this discussion and proves correctness of ignoring states far from the main diagonal.

5.3 Improved Garbled RAM

By replacing stack-based merging with our garbled merge data structure, we obtain the following key theorem.

Theorem 3 (Switch System RAM)**.** *Let P be a RAM program halting in n steps and manipulating w-bit words. There exists an oblivious switch system C such that for any input x, $C(x) = P(x)$. The system C has $O((w + \lg^2 n) \cdot n \lg n) \cdot \omega(1)$ joins.*

Proof. Recall that our GRAM is the Tri-State Circuit GRAM where we replace stacks by our merge component. Recall also (from Section 3.4) that every tri-state circuit join gate can be replaced by a switch system join. The total number of join gates in our Switch System GRAM can be attributed to three sources:

Merge joins. As discussed in Section 3.4, our merge gates contribute a total of $O(wn \lg n)$ join gates.

Joins from non-merge gates. The non-stack joins in Tri-State GRAM arise from the read/write circuitry needed to obliviously read/rearrange the contents of nodes on root-to-leaf paths of the ORAM tree. Each RAM operation requires a constant number of such traversals. In the underlying Circuit ORAM construction, each internal tree node holds $O(1)$ data items, each of size $O(w)$. Since paths are of length $O(\lg n)$, traversing a path consumes $O(w \lg n)$ join gates. An additional $\omega(1)$ term is incurred from the root node and leaf nodes, each of which hold a larger number of $O(\lg n) \cdot \omega(1)$ elements; this is needed to achieve negligible probability of failure Thus, touching any such node incurs $O(wn \lg n) \cdot \omega(1)$ joins, which is the asymptotic bottleneck.

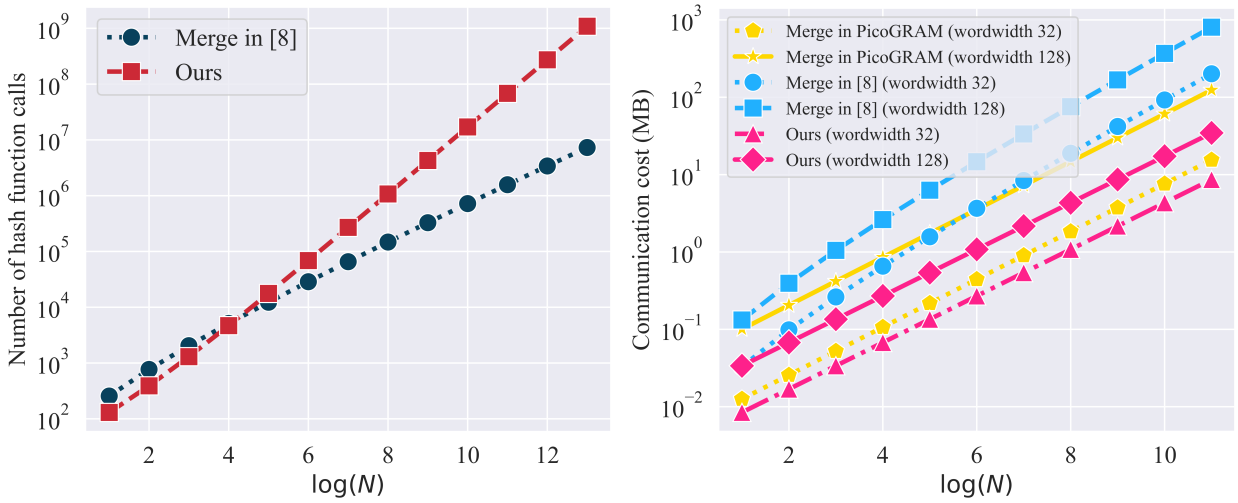
Joins for recursive position maps. As in many ORAM constructions, Tri-State GRAM requires a recursion step. In particular, it uses a recursively instantiated *position map*. This position map tracks where in the ORAM binary tree structure each logical element resides. The number of required levels of recursion is $O(\lg n)$. Note that regardless of the top-level ORAM's word size, the position map uses a word size of $\Theta(\lg n)$ bits. Hence, the total cost of the recursive position map is $O(n \lg^3 n) \cdot \omega(1)$ joins.

Adding the contributions from merge gates, non-merge gates, and recursive position maps gives a total of $O((w + \lg^2 n) \cdot n \lg n) \cdot \omega(1)$ join gates, as claimed. □

The following holds by combining Theorems 1 and 3:

Corollary 1 (Garbled RAM)**.** *There exists a garbling scheme for RAM programs such that for any program P halting in n steps and manipulating w-bit words, the encoding $\tilde{P}$ has the following size:*

$$O((w + \lg^2 n) \cdot n \lg n \cdot \lambda) \cdot \omega(1) \text{ bits}$$



(a) Computation cost

(b) Communication cost

Figure 9: Cost comparison for garbled merging vs. result array size N. (a) Computation cost measured in hash function calls. (b) Communication cost (MB) under different bandwidth settings. Our approach outperforms both prior work in communication cost, at the trade-off of a moderate increase in computation.

6 Evaluation

We implement a prototype of our garbled merge technique and an end-to-end Garbled RAM using our garbled stack in C++, and use the EMP Toolkit to instantiate the underlying hash function, which is based on fixed-key AES [WMK16]. All experiments are run on AWS c5ad.8xlarge machine with 64GB RAM. The security parameter of our scheme is fixed as $\lambda = 128$.

We evaluate performance along two dimensions: communication and computation. Communication cost is measured by the size of garbled programs, while computation cost is measured by the number of AES invocations, which corresponds to the number of evaluated switches.

Highlights. Assuming only the existence of circular-correlation-robust hash functions, our garbled merging achieves up to 3× lower communication cost than PicoGRAM, which requires the stronger DDH assumption. When integrated into GRAM, our approach achieves lower communication costs than PicoGRAM for program sizes from 2^{10} to 2^{24}.

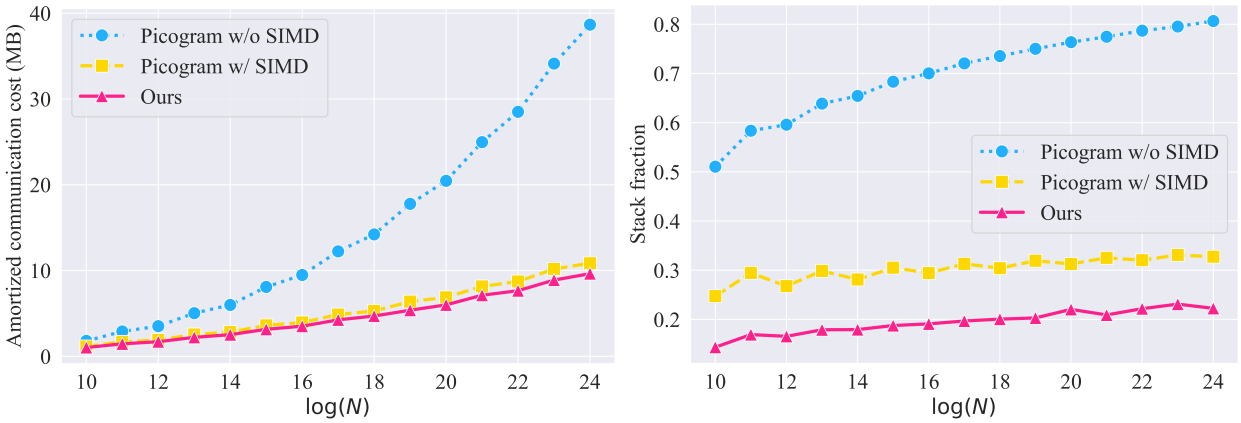
6.1 Garbled Merging Evaluation

We evaluate the communication and computation costs for varying values of n, the length of the merged output array.

Figure 9b confirms that our approach achieves communication cost linear in n and w. At $w = 128$ and $n = 2^{10}$, our scheme requires only 2.1 MB compared to over 44 MB for the Boolean baseline—a 21× reduction. Figure 9a shows the computation cost in hash function calls, reflecting the trade-off in our construction: improved communication at the cost of $\Theta(wn^2)$ computation.

Comparison against [Goo11]. A garbled Boolean-circuit-based compaction network for merging in the style of Goodrich [Goo11] is suggested for the GRAM setting by [GTRS25]. We implement the network with *privacy-free* "half" AND gates [ZRE15]. Our scheme consistently achieves a communication cost roughly 10× lower than this approach (see Figure 9b). Concretely, at $n = 2^{11}$ and $w = 32$, the Boolean-circuit-based merging requires approximately 24 MB, whereas ours requires only approximately 1 MB.

We intentionally trade higher local computation for reduced communication. Our evaluation shows that the computation cost of our approach is roughly two orders of magnitude higher than that of the Boolean



(a) Amortized communication cost

(b) Ratio of stack operation cost

Figure 10: Communication cost comparison per access as a function of $\log(N)$. (a) Our method achieves a smaller amortized communication cost per access than PicoGRAM [GTRS25] (with and without SIMD). (b) Fraction of communication cost attributed to stack operations. Our approach maintains a consistently lower stack ratio (≈20%) than PicoGRAM (≈25–34%), indicating the improvement achieved by our scheme.

network in [Goo11] at $n = 2^{12}$ (see Figure 9a).

Comparison against [GTRS25] PicoGRAM [GTRS25] also provides a compaction network for merging using single-instruction-multiple-data (SIMD) gates. We instantiate the DDH group with the elliptic curve P-256 for merging. Our scheme consistently achieves a lower communication cost than PicoGRAM (see Figure 9b). Concretely, at $n = 2^{11}$ and $w = 128$, PicoGRAM's merging requires approximately 14.8 MB, whereas ours requires only approximately 4.1 MB, achieving 3× improvement.

We do not plot the number of AES calls made by PicoGRAM's DDH-based approach because its main computational cost is group exponentiation. Both techniques use the same compaction network, involving $O(n \lg n)$ gates that conditionally swap two elements. In PicoGRAM, each swap gate incurs w group exponentiations.

6.2 End-to-end Evaluation.

To quantify the impact of our garbled merging primitive on a full GRAM instance, we conducted an end-to-end evaluation by replacing only the stack subroutine in the PicoGRAM construction. Following PicoGRAM's evaluation methodology, we report the communication cost per RAM access as a function of N.

Under this identical experimental setup, replacing the compaction network with our garbled stack yields a consistent constant-factor reduction in end-to-end communication. As shown in Figure 10a, our method achieves lower communication cost than PicoGRAM across all problem sizes $N = 2^{10}$ to 2^{24}, with the curve lying uniformly below that of PicoGRAM. In addition, our method requires only hash functions, avoiding the stronger DDH assumption needed by PicoGRAM.

Figure 10b further shows that our approach maintains a significantly lower stack communication ratio (≈20%) compared to PicoGRAM (≈25-34%). By reducing the stack cost per access by a constant fraction, our construction shifts the contribution to communication from the stack to the non-stack components of the GRAM, demonstrating more efficient bandwidth utilization.

Acknowledgements

This research was supported by NSF grant CNS-#2246353.

References

[BHKR13] Mihir Bellare, Viet Tung Hoang, Sriram Keelveedhi, and Phillip Rogaway. Efficient garbling from a fixed-key blockcipher. In *2013 IEEE Symposium on Security and Privacy*, pages 478–492. IEEE Computer Society Press, May 2013.

[BHR12] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012*, pages 784–796. ACM Press, October 2012.

[CKKZ12] Seung Geol Choi, Jonathan Katz, Ranjit Kumaresan, and Hong-Sheng Zhou. On the security of the "free-XOR" technique. In Ronald Cramer, editor, *TCC 2012*, volume 7194 of *LNCS*, pages 39–53. Springer, Berlin, Heidelberg, March 2012.

[GHL+14] Craig Gentry, Shai Halevi, Steve Lu, Rafail Ostrovsky, Mariana Raykova, and Daniel Wichs. Garbled RAM revisited. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 405–422. Springer, Berlin, Heidelberg, May 2014.

[GKWY20] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. In *2020 IEEE Symposium on Security and Privacy*, pages 825–841. IEEE Computer Society Press, May 2020.

[GLO15] Sanjam Garg, Steve Lu, and Rafail Ostrovsky. Black-box garbled RAM. In Venkatesan Guruswami, editor, *56th FOCS*, pages 210–229. IEEE Computer Society Press, October 2015.

[GLOS15] Sanjam Garg, Steve Lu, Rafail Ostrovsky, and Alessandra Scafuro. Garbled RAM from one-way functions. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th ACM STOC*, pages 449–458. ACM Press, June 2015.

[GO96] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious rams. *Journal of the ACM*, 43(3):431–473, 1996.

[Goo11] Michael T Goodrich. Data-oblivious external-memory algorithms for the compaction, selection, and sorting of outsourced data. In *Proceedings of the twenty-third annual ACM symposium on Parallelism in algorithms and architectures*, pages 379–388, 2011.

[GTRS25] Tianyao Gu, Afonso Tinoco, Sri Harish G. Rajan, and Elaine Shi. PicoGRAM: Practical garbled RAM from decisional Diffie–Hellman. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VIII*, volume 16007 of *LNCS*, pages 247–281. Springer, Cham, August 2025.

[Hea24] David Heath. Efficient arithmetic in garbled circuits. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part V*, volume 14655 of *LNCS*, pages 3–31. Springer, Cham, May 2024.

[HK21] David Heath and Vladimir Kolesnikov. One hot garbling. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 574–593. ACM Press, November 2021.

[HKO22] David Heath, Vladimir Kolesnikov, and Rafail Ostrovsky. EpiGRAM: Practical garbled RAM. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 3–33. Springer, Cham, May / June 2022.

[HKO23] David Heath, Vladimir Kolesnikov, and Rafail Ostrovsky. Tri-state circuits - A circuit model that captures RAM. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 128–160. Springer, Cham, August 2023.

[KS08] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfsdóttir, and Igor Walukiewicz, editors, *ICALP 2008, Part II*, volume 5126 of *LNCS*, pages 486–498. Springer, Berlin, Heidelberg, July 2008.

[LO13] Steve Lu and Rafail Ostrovsky. How to garble RAM programs. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 719–734. Springer, Berlin, Heidelberg, May 2013.

[PLS23] Andrew Park, Wei-Kai Lin, and Elaine Shi. NanoGRAM: Garbled RAM with $\widetilde{O}(\log N)$ overhead. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part I*, volume 14004 of *LNCS*, pages 456–486. Springer, Cham, April 2023.

[Wak68] Abraham Waksman. A permutation network. *Journal of the ACM*, 15(1):159–163, 1968.

[WCS15] Xiao Wang, T.-H. Hubert Chan, and Elaine Shi. Circuit ORAM: On tightness of the Goldreich-Ostrovsky lower bound. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 850–861. ACM Press, October 2015.

[WMK16] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient MultiParty computation toolkit. `https://github.com/emp-toolkit`, 2016.

[Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986.

[ZE13] Samee Zahur and David Evans. Circuit structures for improving efficiency of security and privacy tools. In *2013 IEEE Symposium on Security and Privacy*, pages 493–507. IEEE Computer Society Press, May 2013.

[ZRE15] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole - reducing data transfer in garbled circuits using half gates. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 220–250. Springer, Berlin, Heidelberg, April 2015.

A Garbling Scheme Definition

A garbling scheme is a tuple of procedures that specify how to garble programs. We give (a slight simplification of) the definition as formalized by [BHR12]:

Definition 5 (Garbling Scheme)**.** *A **garbling scheme** $\mathbb{S}$ for a class of programs $\mathcal{P}$ is a tuple of procedures:*

$$\mathbb{S} = (\mathit{Garble}, \mathit{Encode}, \mathit{Eval})$$

- *Garble maps security parameter 1^λ and program $P \in \mathcal{P}$ to garbled program $\tilde{P}$ and an input encoding string e.*
- *Encode maps input encoding string e and cleartext bitstring x to encoded input $\tilde{x}$.*
- *Eval maps program P, garbled program $\tilde{P}$, and encoded input $\tilde{x}$ to an output y.*

*$\mathbb{S}$ is **correct** if the following probability is overwhelming in λ:*

$$\Pr\left[y = P(x) \;\middle|\; \begin{array}{l} (\tilde{P}, e) \leftarrow \mathit{Garble}(1^\lambda, P) \\ \tilde{x} \leftarrow \mathit{Encode}(e, x) \\ y \leftarrow \mathit{Eval}(P, \tilde{P}, \tilde{x}) \end{array} \right] > 1 - \mathit{negl}(\lambda)$$

*$\mathbb{S}$ is **private** if there exists a simulator $\mathcal{S}_{prv}$ such that for any program $P \in \mathcal{P}$ and all inputs x, the following ensembles are indistinguishable in λ:*

$$\left\{ (\tilde{P}, \tilde{x}) \;\middle|\; \begin{array}{l} (\tilde{P}, e) \leftarrow \mathit{Garble}(1^\lambda, P) \\ \tilde{x} \leftarrow \mathit{Encode}(e, x) \end{array} \right\} \stackrel{c}{=} \mathcal{S}_{prv}(1^\lambda, P, P(x))$$

B Circular Correlation Robustness

Roughly speaking, a CCR hash function (CCRH) produces random-looking output, even when hashing inputs related by some correlation Δ, and even when using hash output to encrypt values involving the same correlation Δ. The CCR property enables the crucial Free XOR optimization [KS08], which leverages GC labels related by Δ. Because the property is needed for Free XOR, CCR hashes are heavily leveraged in the symmetric-key GC literature. We recall the definition given by [ZRE15]:

Definition 6 (Circular Correlation Robustness)**.** *We define two oracles that each accept as input a label $K \in \{0,1\}^\lambda$, a bit $x \in \{0,1\}$, and a nonce ν:*

- *$circ_\Delta(K, x, \nu) := H(K \oplus \Delta, \nu) \oplus x \cdot \Delta$ where $\Delta \in_\$ \{0,1\}^{\lambda-1}1$.*
- *$R(K, x, \nu)$ is a uniformly random function with range $\{0,1\}^\lambda$.*

*A sequence of oracle queries (K, x, ν) is **legal** when the same value ν is never queried more than once. H is **circular correlation robust** if no poly-time adversary $\mathcal{A}$ issuing legal queries can distinguish $circ_\Delta$ and R:*

$$\left| \Pr_\Delta \left[\mathcal{A}^{circ_\Delta}(1^\lambda) = 1\right] - \Pr_R \left[\mathcal{A}^R(1^\lambda) = 1\right] \right| < negl(\lambda)$$

C Improving Computation Cost for Randomized Merge

Theorem 4. *Let $s \in \{0,1\}^{2n}$ be drawn uniformly from those strings with Hamming weight n. For each $k \in \{0, ..., 2n\}$, we consider the length-k prefix of s, and we consider the difference between the number of zeros and the number of ones in that prefix:*

$$D_k := \sum_{i=0}^{k} (2s_i - 1)$$

Define the following quantity:

$$\delta := \sqrt{4n(\lambda \lg 2 + \lg(4n))} = O\left(\sqrt{n \cdot \lambda}\right)$$

It is unlikely that any difference D_k will exceed δ:

$$\Pr\left[\max_{0 \le k \le 2n} |D_k| \ge \delta\right] \le 2^{-\lambda}$$

Proof. Fix $k \in \{0, \ldots, 2n\}$ and let

$$X_k \;:=\; \#\{1 \le i \le k :\; s_i = 1\}.$$

Since s is drawn uniformly from strings with n ones and n zeros, we have

$$X_k \sim \text{Hypergeometric}(N = 2n,\; K = n,\; k).$$

Therefore

$$D_k = \sum_{i=1}^{k} (2s_i - 1) = 2X_k - k,$$

and in particular $\mathbb{E}[D_k] = 0$ by symmetry. Equivalently, write $Y_i \in \{-1, +1\}$ for the sequence obtained by sampling without replacement from a multiset with n copies of $+1$ and n copies of -1. Then $D_k = \sum_{i=1}^{k} Y_i$. By Hoeffding's inequality, for every $t > 0$,

$$\Pr(|D_k| \ge t) \;\le\; 2\exp\Big(-\frac{t^2}{2k}\Big).$$

Applying a union bound over $k = 1, 2, \ldots, 2n$ yields

$$\Pr\Big(\max_{0 \le k \le 2n} |D_k| \ge \delta \Big) \;\le\; \sum_{k=1}^{2n} 2 \exp\Big(-\frac{\delta^2}{2k} \Big).$$

The function $k \mapsto \exp\big(-\delta^2/(2k)\big)$ is increasing in k, so each summand is at most the largest one (attained at $k = 2n$). Hence

$$\sum_{k=1}^{2n} 2 \exp\Big(-\frac{\delta^2}{2k} \Big) \;\le\; 4n \exp\Big(-\frac{\delta^2}{4n} \Big).$$

Now choose δ so that

$$4n \exp\Big(-\frac{\delta^2}{4n} \Big) \;\le\; 2^{-\lambda}.$$

Solving for δ gives

$$\delta \;\ge\; \sqrt{4n(\lambda \lg 2 + \lg(4n))}.$$

With this choice of δ, the probability is at most $2^{-\lambda}$, as claimed. □

We use this tail bound as our threshold parameter and denote it by $\delta = \delta_\lambda$, which lies in $\widetilde{O}(\sqrt{n \cdot \lambda})$. We define special-case handling for those states whose i coordinate is exactly δ higher than its j coordinate (resp. j coordinate is exactly δ higher than its i coordinate).

$$q_{i,i-\delta} := q^0_{i-1,i-\delta} \qquad\qquad q_{j-\delta,j} := q^1_{j-\delta,j-1} \tag{8}$$

$$q^1_{i,i-\delta} := q_{i,i-\delta} \qquad\qquad q^0_{j-\delta,j} := q_{j-\delta,j} \tag{9}$$

Crucially, the parties now simply ignore those states i, j where j is significantly higher than i (resp. i is significantly higher than j). Namely, they ignore those states $q_{i,j}$ where $j > i + \delta$ or $i > j + \delta$.

D Garbled Merge Sort

While the main motivation for our merge system is Garbled RAM, the primitive is broadly applicable. For instance, we show that the primitive can be straightforwardly used to achieve a garbled version of merge sort which achieves state of the art communication performance:

Lemma 1 (Garbled Merge Sort)**.** *Let x denote a length-n array of w-bit elements, and let $\le$ denote a relation that induces a total ordering on array elements. Suppose there is a switch system that obliviously computes $\le$. Then there exists an oblivious switch system C with the following characteristics:*

- *C sorts x according to $\le$.*
- *C includes $O(n \lg n)$ copies of the sub-system computing $\le$.*
- *C has an additional $O(w \cdot n \lg n)$ joins.*
- *C has an additional $O(w \cdot n \cdot \sqrt{n \cdot \lambda})$ non-join gates.*

For instance, if $\le$ computes a standard lexicographic ordering, then C has $O(w \cdot n \lg n)$ total joins and $O(w \cdot n \cdot \sqrt{n \cdot \lambda})$ total non-joins.

Proof. By constructing C. C computes the standard mergesort algorithm, where the merge operation is implemented via our switch-system-based merge (Theorem 2). Note that if this were done naïvely, then C would leak the relative order of array elements, as our merge algorithm leaks its control bits. This leakage can be removed by applying the well-known shuffle-then-sort paradigm.

In particular, C first permutes x using a permutation network. A permutation network can be achieved by a Boolean circuit of size $O(w \cdot n \lg n)$ [Wak68]. [Hea24] shows such a circuit can be implemented by a switch system with the same asymptotic number of gates. □

```
Garble(1^λ, (C, D)) :
  Δ ∈$ {0,1}^{λ−1}1   ;   r ∈$ D
  C̃, e ← empty-string
  append Δ to e
  for each input wire w :
    K^0_w ∈$ {0,1}^λ
    append K^0_w to e
  for each random w set to r[i] :
    K^0_w ← r[i] · Δ
  for (g, gid) ∈ C ; match g :
    case reveal(x)
      append lsb(K^0_x) to C̃
    case y := x ⇐ ctrl ?= 0 :
      K^0_y ← K^0_x ⊕ H(K^0_ctrl, gid)
    case x = y :
      append K^0_x ⊕ K^0_y to C̃
    case z := x ⊕ y : K^0_z ← K^0_x ⊕ K^0_y
  for each output wire w :
    append lsb(K^0_w) to C̃
  return (C̃, e)
```

```
S_prv(1^λ, (C, D), y) :
  reveals ← S_reveal(1^λ)
  C̃, x̃ ← empty-string

  for each input wire w :
    K_w ∈$ {0,1}^λ
    append K_w to e
  for each random w :
    K_w ← 0^λ
  for (g, gid) ∈ C ; match g :
    case reveal(x) :
      append lsb(K_x) ⊕ reveals[x] to C̃
    case y := x ⇐ ctrl ?= 0 :
      if reveals[ctrl] = 0 :
        K_y ← K_x ⊕ H(K_ctrl, gid)
      else : K_y ∈$ {0,1}^λ
    case x = y :
      append K_x ⊕ K_y to C̃
    case z := x ⊕ y : K_z ← K_x ⊕ K_y
  for each output wire w :
    let y[w] be output on wire w
    append y[w] ⊕ lsb(K^0_w) to C̃
  return (C̃, x̃)
```

Figure 11: Our procedure for garbling oblivious switch system families (left) and our simulator used to show privacy of our scheme (right). *gid* denotes a gate-specific nonce. *lsb* outputs the lsb of a Boolean GC label. H is a CCRH (Definition 6). We highlight important components of the simulator.

E A Switch System Garbling Scheme

For completeness, we formalize a garbling scheme for switch systems. The scheme presented here is very similar to that of [Hea24]. The main differences are:

- We specify handling of our *reveal* gates; in [Hea24], each switch included an extra bit to reveal its control to E. In the formalization here, we need only one bit per *reveal* gate.
- We specialize handling to the Boolean gates, as opposed to also including arithmetic wires. This somewhat simplifies the scheme.
- We construct a scheme where the evaluator obtains output upon completing the call to *Eval*, rather than including an extra *Decode* step.

We emphasize that our goal here is simply to provide a complete reference—especially in terms of how to handle *reveal*, together with a slightly simpler presentation than that of [Hea24].

Informally, switch system gates are handled as follows:

```
1  Eval((C, D), C̃, x̃) :
2    for each input wire w :
3      fetch K_w from x̃
4    for each random w set to r[i] :
5      K_w ← 0^λ
6    reveals ← empty
7    for (g, gid) ∈ Ready ; match g :
8      case reveal(x)
9        fetch lsb(K_x^0) from C̃
10       reveals[x] ← lsb(K_x) ⊕ lsb(K_x^0)
11     case y := x ⇐ ctrl ?= 0 :
12       if reveals[ctrl] = 0 :
13         enforce K_y = K_x ⊕ H(K_ctrl, gid)
14       else : do nothing
15     case x = y :
16       fetch K_x^0 ⊕ K_y^0 from C̃
17       enforce K_y = K_x ⊕ (K_x^0 ⊕ K_y^0)
18     case z := x ⊕ y :
19       enforce K_z = K_x ⊕ K_y
20   y ← empty-string
21   for each output wire w :
22     fetch lsb(K_w^0) from C̃
23     append lsb(K_x) ⊕ lsb(K_x^0) to y
24   return y
```

```
1  Encode(e, x) :
2    x̃ ← empty-string
3    fetch Δ from e
4    for x_i ∈ x
5      fetch K_{x_i}^0 from e
6      append K_{x_i}^0 ⊕ x_i · Δ to x̃
7    return x̃
```

Figure 12: Our procedure for evaluation (left) and encoding circuit inputs (right). In *Eval*, the evaluator implicitly maintains a set of gates *Ready*, which contains those gates for which some connected wire label may be able to be solved. Initially, *Ready* contains those gates connected to input wires; when the evaluator uses a gate to solve a wire label, she adds all other gates connected to that wire to *Ready*. Syntax **enforce** denotes that the evaluator ensures some equation on wire labels holds; for example, they might use two wires connected to an XOR to solve for the third. (Definition 6).

Switches Garbling a switch is relatively straightforward. Roughly, each wire's logical value is encoded by one of two possible encryption keys – one corresponding to logical zero, the other to logical one. To garble a switch, we use the control wire's zero key to encrypt (using a CCRH H) the difference between x keys and y keys. At evaluation and if the control wire holds zero, E can use its control wire key to decrypt this difference, allowing it to translate between x keys and y keys. Note that it therefore must be the case that E *knows* whether the control wire holds zero or one, since she must know whether she should attempt to translate or not.

XORs XOR gates in switch systems closely resemble those in Boolean circuits and employ the classic Free XOR technique [KS08]. The garbler selects a uniformly random[5] global offset Δ, ensuring that for each wire, the one-key equals the zero-key XORed with Δ.

To garble an XOR gate with input wires x, y, and output wire z, the garbler sets the zero-key of z as the XOR of the zero-keys of x and y; the one-key is derived by adding Δ. This introduces a linear relationship among the wire keys. During evaluation, the evaluator simply XORs the input keys to obtain the output key. No ciphertexts from the garbler are required, so XOR gates incur zero communication cost. Moreover, XOR gates in switch systems support multidirectional evaluation: since $x \oplus y = z$ implies $x \oplus z = y$ and $y \oplus z = x$, the evaluator can compute any one of the three wire values given the other two.

[5] Δ is sampled uniformly over a *subset* of $\{0,1\}^\lambda$, namely $\{0,1\}^{\lambda-1}1$. This ensures that the zero-key and one-key differ in the least significant bit (LSB).

Joins In a switch system, each non-input wire is *defined* by exactly one switch/XOR, and each switch/XOR defines its output wire. Switches/XORs contribute *nothing* to the size of the garbled program, and in this sense they are "free". However, with only switches and joins, we cannot construct arbitrary circuits, as we will sometimes need to assert that the output of two different XORs/switches are "the same wire", and our restriction that each gate defines a wire prevents this. Therefore, we need a special mechanism – join – by which we can assert that two wires are, in fact, the same.

Consider a join $x = y$ and suppose x's zero key is K_x; y's zero key is K_y. To garble such a join, G includes in the garbled circuit the difference between keys $K_x \oplus K_y$. This *translation value* allows the evaluator to convert an x key into a y key, and vice versa. Each join adds λ bits to the size of the garbled program, and joins are the main source of communication cost. *Indeed, a primary goal in switch system design is to minimize the number of joins.*

Construction 1 (Garbled Switch Systems)**.** *Figures 11 and 12 specify a garbling scheme* $(\mathit{Garble}, \mathit{Encode}, \mathit{Eval})$ *that handles computations expressed as oblivious switch systems (Definition 1).*

Correctness of the scheme is immediate by inspection of Figures 11 and 12 and the semantics of switch systems as specified in Section 4.

Lemma 2. *Construction 1 is* ***correct*** *(Definition 5). Namely, let* (C_λ, D_λ) *denote an oblivious switch system that computes* f*. The following probability is overwhelming in* λ*:*

$$\Pr\left[y = f(x) \;\middle|\; \begin{array}{l} (\tilde{C}, e) \leftarrow \mathit{Garble}(1^\lambda, (C_\lambda, D_\lambda)) \\ \tilde{x} \leftarrow \mathit{Encode}(e, x) \\ y \leftarrow \mathit{Eval}((C_\lambda, D_\lambda), \tilde{C}, \tilde{x}) \end{array} \right] > 1 - \mathit{negl}(\lambda)$$

Lemma 3. *If the hash function* H *is a CCRH (Definition 6), then Construction 1 is* ***private*** *(Definition 5). Namely, let* (C_λ, D_λ) *denote an oblivious switch system that computes* f*. The following holds:*

$$\left\{ (\tilde{P}, \tilde{x}) \;\middle|\; \begin{array}{l} (\tilde{C}, e) \leftarrow \mathit{Garble}(1^\lambda, (C_\lambda, D_\lambda)) \\ \tilde{x} \leftarrow \mathit{Encode}(e, x) \end{array} \right\} \stackrel{c}{=} \mathcal{S}_{prv}(1^\lambda, (C_\lambda, D_\lambda), f(x))$$

Proof. By construction of a privacy simulator $\mathcal{S}_{prv}$, as specified in Figure 11. The argument for indistinguishability is similar to the obliviousness proof given in [Hea24], so we proceed at a high level.

The goal is to show that the $\mathcal{S}_{prv}$ is indistinguishable from the evaluator's real-world view of the garbled program. The main differences between the real-world garbled circuit and the simulation are highlighted in Figure 11:

- The revealed values are sampled by the switch system's obliviousness simulator $\mathcal{S}_{reveal}$, as opposed to depending on real-world values. Ultimately, this handling is appropriate due to the obliviousness of the switch system: the real-world leakage is indistinguishable from the simulated leakage.
- The simulator arranges that reveal gates (resp. output wires) reveal bits consistent with the leakage as specified by $\mathcal{S}_{reveal}$ (resp. the true output $y = f(x)$).
- The handling of a switch gate depends on the value of that switch's control (which must be deducible). If the switch is open (i.e., the control has value 1), then the simulator samples the output data wire's label as a uniform string $K_y \in_\$ \{0,1\}^\lambda$. Ultimately, this handling is appropriate because H is a CCRH.

Indistinguishability of this simulation can be argued by a hybrid argument essentially identical to the one given by [Hea24]. □